

Bloque CRUD extra — Edición y borrado

Sistema de Gestión de Torneos Deportivos de Contacto (Taekwondo)

Alcance: trabajo realizado entre H0004 (Cargar competidores) y H0005 (Generar llaves).

Naturaleza: no es una historia de usuario del backlog — es un bloque de completitud del CRUD pedido explícitamente por el usuario antes de encarar H0005.

Rama: Dev · **Fecha del documento:** 2026-08-19

1. Por qué existió este bloque

Al terminar H0004 el sistema podía *crear* torneos, categorías y competidores, y *editar* torneos, pero no había forma de deshacer un error de carga. En un torneo real la carga de competidores se hace contra reloj y con datos dictados por teléfono o WhatsApp: un peso mal tipeado o un competidor duplicado son la norma, no la excepción.

Además H0005 depende directamente de la calidad de esos datos: la clasificación automática usa **sexo, edad, peso y graduación** para decidir en qué categoría entra cada competidor. Sin poder corregir esos campos, un error de tipeo se propagaba a las llaves sin remedio. Por eso el bloque se hizo **antes** de H0005 y no después.

2. Qué se implementó

2.1 Operaciones nuevas

Operación	Ruta	Rol	Respuesta OK
PUT Editar competidor	/api/v1/torneos/{torneoId}/competidores/{id}	Coordinador	200 + CompetidorResponse
DELETE Eliminar competidor	/api/v1/torneos/{torneoId}/competidores/{id}	Coordinador	204 No Content
DELETE Eliminar categoría	/api/v1/torneos/{torneoId}/categorias/{id}	Coordinador	204 No Content
DELETE Eliminar torneo	/api/v1/torneos/{id}	Coordinador	204 No Content

Nota: **PUT** Editar categoría y **PUT** Editar torneo ya existían (H0002 y H0003 respectivamente), pero **sus guards se endurecieron** en este bloque para quedar consistentes con las reglas de borrado.

2.2 Estructura por capa (patrón repetido en las 4 operaciones)

Cada operación siguió la misma vertical de Clean Architecture + CQRS, sin excepciones:

Capa	Artefacto
API	<code>Endpoints/{Entidad}/{Verbo}{Entidad}Endpoint.cs</code> — FastEndpoints, <code>Roles("Coordinador")</code>
API	<code>{Verbo}{Entidad}Request.cs</code> (+ <code>Validator</code> cuando hay body)
Application	<code>Features/{Entidad}/Commands/{Verbo}{Entidad}/</code> — Command, Handler y Validator
Domain	<code>Interfaces/I{Entidad}Repository</code> — se agregaron <code>DeleteAsync</code> / <code>UpdateAsync</code>
Infrastructure	<code>Persistence/Repositories/{Entidad}Repository.cs</code> — implementación EF Core

3. Decisiones de diseño

D1 — Borrado físico, no soft-delete

Se descartó el patrón `deleted_at` / `IsDeleted`. Razones:

- El dominio no tiene requisito de auditoría ni de "papelera de reciclaje": un competidor cargado por error simplemente no debería existir.
- El soft-delete obliga a filtrar por `IsDeleted` en **todas** las queries — incluido el clasificador de H0005. Un olvido ahí mete competidores fantasma en un bracket, que es el peor bug posible del sistema.
- El borrado real está acotado por los guards de estado (D2), que ya impiden destruir datos de un torneo en juego.

Contrapartida asumida: no hay "deshacer" del lado del servidor. La red de contención es la confirmación explícita en la UI (ver §5) y los guards de estado.

D2 — Guards por estado del torneo, no por permisos

Todos los guards son **del dominio**, no de autorización: no dependen de *quién* pide la operación (eso ya lo resuelve `Roles("Coordinador")`) sino de *en qué momento del ciclo de vida* está el torneo. Se implementan en el Command Handler, nunca en el endpoint.

Operación	Guard	HTTP	Mensaje
Eliminar torneo	Estado debe ser Borrador	409	"Solo se pueden eliminar torneos en estado Borrador."
Editar / eliminar categoría	Torneo no Finalizado	409	"No se pueden {editar eliminar} las categorías de un torneo finalizado."
Editar / eliminar categoría	Sin llaves generadas (<code>LlavesGeneradas == false</code>)	409	"No se puede {editar eliminar} una categoría con llaves generadas."
Editar / eliminar competidor	Torneo no Finalizado	409	"No se pueden {editar eliminar} {los competidores competidores} de un torneo finalizado."
Todas	El recurso debe pertenecer al torneo de la ruta	404	<code>NotFoundException</code>

Sobre el guard de pertenencia (404): todos los handlers verifican `entidad.TorneoId != request.TorneoId` y lanzan 404, no 403. Es deliberado: responder "existe pero no es de este torneo" filtraría la existencia de recursos de otros torneos. El 404 es indistinguible de un id inventado.

Guards latentes: hoy **no existe transición de estados** del torneo — no hay endpoint para pasar de Borrador a Activo o Finalizado, y generar llaves tampoco cambia el estado. Todos los torneos quedan en `Borrador`. Consecuencia: los guards de "Finalizado" son *correctos pero inalcanzables* en la versión actual. Están escritos y testeados a la espera de la historia de transición de estados.

D3 — El guard de `LlavesGeneradas` protege la integridad del bracket

Es el guard más importante del bloque y el que conecta con H0005. Una vez que una categoría tiene llaves generadas, editarla (cambiar sus rangos) o eliminarla dejaría un bracket huérfano o inconsistente con los competidores que lo componen. Por eso la categoría queda **congelada**.

Esto se refuerza con la regla de H0005 de **no regenerar llaves**: una categoría con `LlavesGeneradas = true` no se vuelve a generar.

D4 — Comportamiento de las claves foráneas

Relación	ON DELETE	Efecto
Torneo → Categorías	CASCADE	Borrar el torneo borra sus categorías
Torneo → Competidores	CASCADE	Borrar el torneo borra sus competidores
Torneo → Llaves	CASCADE	Borrar el torneo borra sus brackets

Relación	ON DELETE	Efecto
Categoría → Competidores	SET NULL	Borrar la categoría <i>no</i> borra competidores: quedan sin categoría
Competidor → Llaves	RESTRICT	Impide a nivel BD borrar un competidor que está en un match

El **SET NULL** de categoría → competidores es coherente con la decisión de H0004/H0005 de que `categoria_id` sea **nullable**: el competidor se carga sin categoría y se le asigna una al generar las llaves. Borrar una categoría simplemente devuelve a sus competidores al estado "sin clasificar", listos para volver a clasificarse. No se pierde ningún dato de inscripción.

D5 — Editar competidor no cambia su categoría

`UpdateCompetidorCommandHandler` actualiza los datos personales y deportivos (nombre, apellido, sexo, edad, graduación, peso, altura, escuela, responsable, teléfono) pero **nunca toca** `CategoriaId`. La asignación de categoría es responsabilidad exclusiva del clasificador de H0005. Mantener un único dueño de ese campo evita que la UI y el algoritmo se pisen.

D6 — Reglas de validación compartidas entre Command y Request

El validator del `Request` (capa API) y el del `Command` (capa Application) necesitan las mismas reglas, y duplicarlas garantizaba que se desincronizaran. Se resolvió con **interfaz + extension method**:

- `ICompetidorData` + `CompetidorValidationRules.AddCompetidorRules()`
- `ICategoriaData` + `CategoriaValidationRules.AddCategoriaRules()`

Tanto el Request como el Command implementan la interfaz, y cada validator invoca `this.Add{Entidad}Rules()`. Una sola definición de las reglas, dos puntos de aplicación.

4. Detalle por operación

4.1 Eliminar torneo

El más destructivo del sistema: por CASCADE arrastra categorías, competidores y llaves. De ahí que sea el único con el guard más estricto (**solo Borrador**), en lugar del "no Finalizado" del resto. Un torneo que ya arrancó no se borra: se termina.

4.2 Eliminar categoría

Doble guard: torneo no Finalizado y categoría sin llaves generadas. Sus competidores sobreviven con `categoria_id = NULL` (D4), lo que hace de esta operación la vía práctica para **rehacer una clasificación** mal planteada... siempre que todavía no se hayan generado las llaves.

4.3 Eliminar competidor

Guard de torneo no Finalizado. La FK **RESTRICT** hacia `llaves_competencia` protege a nivel base de datos, pero **no hay guard aplicativo equivalente** — ver §7.

4.4 Editar competidor

Guard de torneo no Finalizado, más las reglas compartidas de validación (D6). Es la operación que da sentido a todo el bloque: permite corregir el peso, la edad o la graduación **antes** de generar las llaves, que es exactamente lo que la clasificación de H0005 va a leer.

5. Frontend

5.1 Componente `ConfirmDialog`

Se creó `components/ui/confirm-dialog.tsx` (sobre Base UI `AlertDialog`) como componente reutilizable para **todo** borrado del sistema. Lo consumen:

- `components/torneos/TorneoCard.tsx`
- `components/categorias/CategoriaCard.tsx`
- `components/competidores/CompetidorList.tsx`

Responde a la regla de prevención de errores de `skills/ux-ui-guidelines.md`: toda acción irreversible exige confirmación explícita o Undo. Como el backend hace borrado físico (D1), la confirmación es la única red de contención.

5.2 Tratamiento visual del botón "Eliminar" (regla 60-30-10)

Los botones de eliminar van **atenuados en reposo** (`text-muted-foreground`) y **pasan a rojo en hover / focus**.

El motivo es la regla 60-30-10 del proyecto: el rojo está reservado al 10% de la interfaz, para los CTAs principales (ej. "Declarar ganador"). Pintar de rojo permanente cada botón de borrar en una lista de 15 competidores haría que el color perdiera todo su valor de señal — y encima destacaría visualmente la acción más peligrosa. Atenuado en reposo, la intención destructiva se comunica en el momento en que el usuario apunta al botón.

5.3 Hooks de TanStack Query

Un hook de mutación por operación, todos invalidando las queries relevantes en `onSuccess`:

Archivo	Hooks agregados
<code>hooks/useTorneos.ts</code>	<code>useEditarTorneo</code> , <code>useEliminarTorneo</code>
<code>hooks/useCategorias.ts</code>	<code>useEditarCategoria</code> , <code>useEliminarCategoria</code>
<code>hooks/useCompetidores.ts</code>	<code>useEditarCompetidor</code> , <code>useEliminarCompetidor</code>

En la UI, las acciones bloqueadas por un guard de backend **se ocultan** en lugar de mostrarse y fallar: una categoría con llaves generadas no muestra sus botones de editar/eliminar. El guard del servidor sigue siendo la autoridad — el ocultamiento es solo UX.

5.4 Bug de infraestructura resuelto en este bloque

Los `DELETE` fueron los primeros requests **sin body** del sistema, y destaparon un bug en el fetch wrapper: mandaba `Content-Type: application/json` siempre. FastEndpoints intentaba entonces deserializar un cuerpo vacío y devolvía `400 "One or more errors occurred!"`.

Fix: `lib/api.ts` ahora envía `Content-Type` solo cuando hay body. Se agregó además `extraerMensajeError()`, porque FastEndpoints devuelve `errors` como **objeto** (`{campo:[msg]}`) mientras que el middleware propio lo devuelve como arreglo; usar `body.message` a secas mostraba siempre el genérico.

Ambas reglas quedaron fijadas como reglas 9 y 10 de `frontend/CLAUDE.md` para no reintroducirlas.

6. Tests

Cada handler nuevo tiene su suite de tests unitarios (xUnit + NSubstitute):

- `Features/Torneos/DeleteTorneoCommandHandlerTests.cs`
- `Features/Categorias/DeleteCategoriaCommandHandlerTests.cs`
- `Features/Categorias/UpdateCategoriaCommandHandlerTests.cs`
- `Features/Competidores/DeleteCompetidorCommandHandlerTests.cs`
- `Features/Competidores/UpdateCompetidorCommandHandlerTests.cs`

Los escenarios cubiertos por handler son: camino feliz, recurso inexistente (404), recurso de otro torneo (404) y cada guard de estado (409).

Estado de la suite: 90/90 tests en verde (`cd backend && dotnet test`).

7. Deuda técnica conocida

Pendiente	Detalle
Guard aplicativo al borrar un competidor en un bracket	La FK <code>llaves_competencia → competidores</code> es <code>RESTRICT</code> , así que la BD rechaza el borrado, pero sin guard en el handler eso sale como error crudo de EF Core en vez de un 409 con mensaje claro. Latente hasta que se generen llaves con datos reales.
Borrar / regenerar llaves	No existe. Si la clasificación de H0005 queda mal, no hay forma de rehacerla: la categoría queda congelada por el guard de <code>LlavesGeneradas</code> (D3). Candidato fuerte a historia futura.
Transición de estados del torneo	Sin endpoint Borrador → Activo → Finalizado. Activaría todos los guards latentes de D2.
Validación de encaje al editar competidor	Hoy la clasificación se valida solo al generar llaves. Editar un competidor a valores que no encajan en ninguna categoría no avisa nada en el momento.
Nombres de propiedad en errores de rangos	Aparecen como <code>RangoEdadMin.Value</code> por los tipos nullable. Cosmético.

